PROTOCOLO WEB

# HTTP

# HTTP

*HyperText Transfer Protocol*

*El lenguaje de la Web*

Emmanuel Cetina Gomez

# Arquitectura Cliente-Servidor

El cliente (navegador) envía una petición HTTP. El servidor la recibe, procesa y responde. La comunicación viaja a través de una conexión TCP/IP establecida entre ambos.

## SOCKET

Un Socket es la combinación de una dirección IP y un número de puerto. Actúa como el punto de origen o destino de cualquier conexión de red. Para que el cliente se conecte al servidor HTTP, necesita su IP (o dominio que el DNS traduce) y el puerto donde el servidor "escucha" — por defecto el puerto 80 para HTTP y el 443 para HTTPS.

CLIENTE

### Navegador Web

Inicia la comunicación. Envía peticiones HTTP. Puede ser un browser, app móvil o dispositivo IoT.

SOCKET

### IP + Puerto

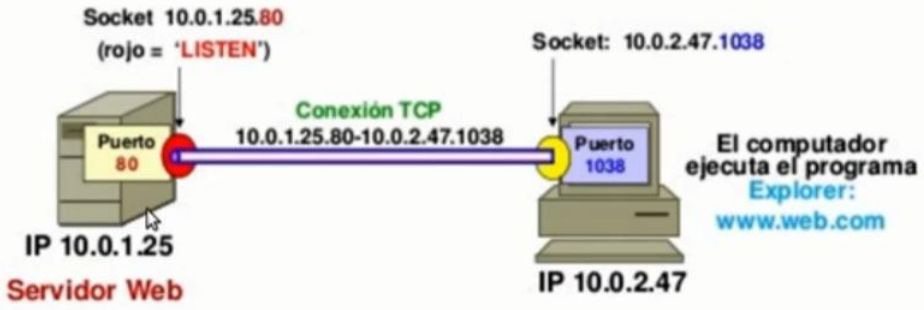
Ej: IP 10.0.1.25 · Puerto 80. El servidor escucha en ese socket. El DNS traduce dominios a IPs.

SERVIDOR

### Servidor Web

Escucha continuamente. Recibe, procesa y responde peticiones. Puede ser cualquier equipo en red.

# 1. Arquitectura cliente/servidor



# ¿Qué es HTTP? — 6 Características Fundamentales

HyperText Transfer Protocol — protocolo que define las reglas y el formato de los mensajes entre navegador y servidor. Es la columna vertebral de internet, escrito en texto plano legible por humanos.

## 01 Cliente-Servidor

Un cliente hace la petición, un servidor la responde. Siempre.

## 02 Sin Estado (Stateless)

Cada petición es independiente. El servidor no recuerda la anterior.

## 03 Capa de Aplicación

Funciona sobre TCP, que gestiona el transporte real de los datos.

## 04 Métodos / Verbos

GET · POST · PUT · PATCH · DELETE · HEAD · OPTIONS

## 05 Códigos de Estado

1xx Info · 2xx Éxito · 3xx Redirección · 4xx Error cliente · 5xx Error servidor

## 06 Cabeceras (Headers)

Metadata en pares clave-valor que acompaña peticiones y respuestas.

# URL / URI — La Dirección de los Recursos

```
https ://  usuario:pass  @  ejemplo.com  :  8042  /  ruta/recurso  ?  param=valor  #
seccion
```

ESQUEMA

### http / https

Protocolo. https = HTTP encriptado con TLS. Separado por ://

AUTORIDAD

### Dominio o IP + Puerto

Identifica el servidor. Sin puerto, HTTP usa 80 y HTTPS usa 443.

RUTA (PATH)

### /carpeta/archivo

Ubicación del recurso. "/" entrega el índice (index.html por defecto).

QUERY STRING

### ?param=valor

Parámetros opcionales separados con &. Solo en GET para filtrar datos.

FRAGMENTO

### #sección

Solo lo usa el navegador para saltar a un lugar. No se envía al servidor.

SOCKET

### IP + Puerto = Destino

El DNS convierte el dominio a IP. IP + Puerto forman el socket de destino.

nombre_de_esquema : parte_jerárquica [ ? consulta ] [ # fragmento ]

# URI - Ejemplos

```
  foo://username:password@example.com:8042/over/there/index.dtb?type=animal&name=narwhal#nose
  \_/   \_______________/ \_________/ \__/            \___/ \_/ \______________________/ \__/
   |           |               |       |                |    |            |                |
   |       userinfo         hostname  port              |    |          query          fragment
   |    \_______________________________/\_____________|____|/ \__/        \__/
   |                    |                        |     |    |    |          |
scheme              authority                   path   |    |    interpretable as keys
 name   \_________________________________________|____|/      \____/      \_____/
   |                         |                    |    |         |           |
   |                 hierarchical part            |    |    interpretable as values
   |                                              |    |
   |            path               interpretable as filename |
   |   ___________|____________                             |
  / \ /                        \                             |
  urn:example:animal:ferret:nose               interpretable as extension
```

# HTTP Request — La Petición del Cliente

## ESTRUCTURA DE LA PETICIÓN

### ① LÍNEA DE REQUEST

```
GET /index.html HTTP/1.1
```

Método + Ruta + Versión HTTP

### ② CABECERAS (HEADERS)

```
Host: www.ejemplo.com
Accept: text/html
```

Pares clave-valor. Host es obligatorio en HTTP/1.1.

### ③ LÍNEA EN BLANCO

```
↵  (fin de cabeceras)
```

Indica que terminaron los headers.

### ④ CUERPO (BODY)

```
quantity=1&price=449
```

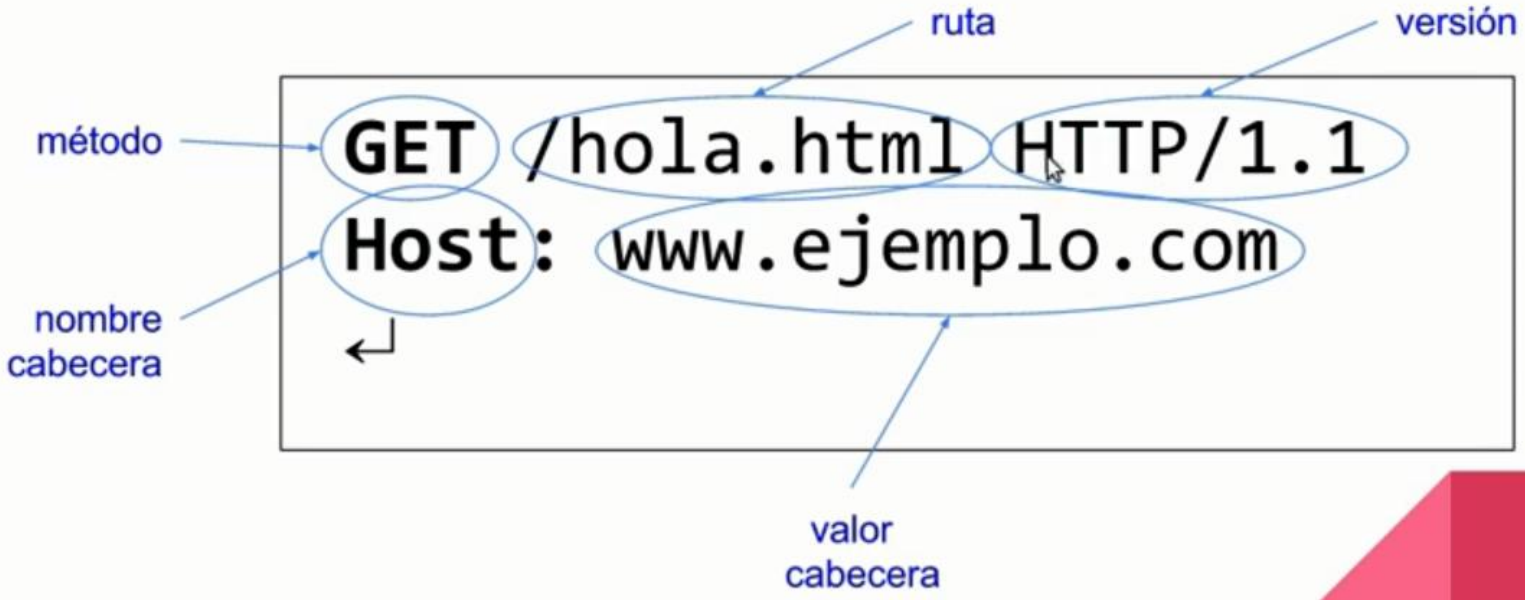
Opcional. Solo en POST/PUT para enviar datos.

## GET vs. POST

| | GET | POST |
|---|---|---|
| Datos | En la URL | En el cuerpo |
| Propósito | Leer / recuperar | Crear / modificar |
| Longitud | Limitada (URL) | Sin límite |
| Caché | ✓ Se cachea | ✗ No se cachea |
| Idempotente | ✓ Sin efectos | ✗ Puede crear algo |

# HTTP request

http://www.ejemplo.com/hola.html



# HTTP Response — La Respuesta del Servidor

## EJEMPLO DE RESPUESTA

```
HTTP/1.1  200 OK                          ← línea de response
Date: Tue 2012 04:33:33 GMT
Server: Apache/2.2.3                      ← cabeceras
Content-Type: text/html; utf-8
Content-Length: 1539
↵
<html>...</html>  (1539 bytes)            ← cuerpo (body)
```

## PARTES DE LA RESPUESTA

**Línea de Response**
Versión HTTP + Código de estado (ej. 200 OK).

**Cabeceras**
Content-Type, Content-Length, Server, Cache-Control...

**Línea en blanco**
Separación entre cabeceras y cuerpo.

**Cuerpo (Body)**
El recurso: HTML, JSON, imagen, PDF, etc.

## CÓDIGOS DE ESTADO

| 1XX | 2XX | 3XX | 4XX | 5XX |
|---|---|---|---|---|
| **Información** | **Éxito** | **Redirección** | **Error cliente** | **Error servidor** |
| 100 Continue | 200 OK · 204 No Content | 301 Moved · 302 Found | 404 Not Found · 401 Unauth. | 500 Internal · 503 Unavail. |

# HTTP response

línea de response

versión

código de estado

cabeceras

contenido
(cuerpo)

```
HTTP/1.1 200 OK
Date: Tue mar 2012 04:33:33 GMT
Server: Apache /2.2.3
Content-Type: text/html; charset=utf-8
Content-Length: 1539
↵
blablabla... (así hasta completar los 1539 bytes)
```